EE675: Introduction to Reinforcement Learning

Assignment 1 - Question 2

Rudransh Verma (230881)

April 13, 2026

1 Working Principles

This assignment explores the implementation and efficiency of classical Reinforcement Learning algorithms in a high-dimensional predator-prey grid world.

1.1 Value Iteration

The `value_iteration` function computes the optimal Q-function by repeatedly applying the Bellman Optimality Operator.

- **Transition Logic**: We utilize a sparse transition kernel $\mathcal{P}$ of size $|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}|$. The update follows: $Q_{k+1}(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a)V_k(s')$.
- **Efficiency**: By flattening the reward and performing sparse matrix-vector multiplication, we avoid nested loops, critical as the state space grows at N^4.

1.2 Policy Iteration

The `policy_iteration` function alternates between evaluation and improvement steps until stability.

- **Iterative Policy Evaluation**: To avoid $O(|\mathcal{S}|^3)$ matrix inversion, we implement a recursive method, iterating $V_{k+1} = R_\pi + \gamma P_\pi V_k$ until convergence.
- **Policy Improvement**: We greedily update the policy. Using integer-based indexing ensures numerical stability and rapid convergence compared to matrix-based updates.

1.3 Greedy Policy Extraction

The `induced_policy` function extracts a deterministic policy from the Q-function by identifying the action index that maximizes $Q(s,\cdot)$ for every state s.

2 Optimization Strategy and Implementation Notes

The computational scale of $N = 12$ requires optimizations to remain within memory and time limits.

2.1 Direct Slicing vs. Induced Functions

In Question 2, we utilized Direct Slicing instead of the formal `induced_kernel` functions.

- **The Bottleneck**: Formally inducing a kernel involves a "Selector Matrix" of size $|\mathcal{S}| \times (|\mathcal{S}| \cdot |\mathcal{A}|)$. At $N = 12$, even sparse allocation creates a performance bottleneck.
- **The Solution**: Since the policies used in Policy Iteration are deterministic, we use index-based slicing (`kernel[row_indices, :]`) to extract the relevant transitions. This is a pointer-level operation that bypasses the computational overhead of sparse matrix multiplication, significantly reducing runtime.

2.2 Recursive Evaluation

Recursive evaluation scales linearly with the non-zero elements in the sparse kernel, making the N^4 state space tractable where inversion would fail.

3 Results

3.1 L_1-Difference and Runtime Performance

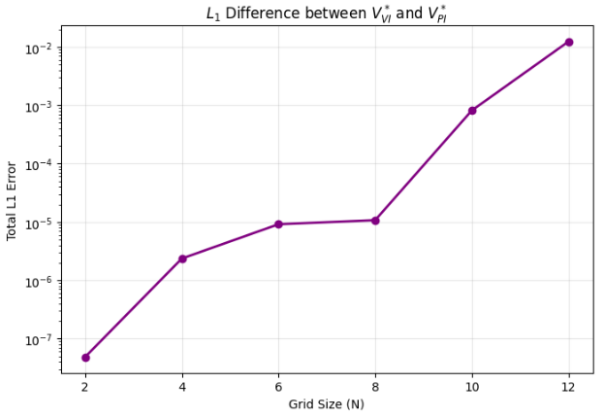


Figure 1: L_1-diff between VI and PI.

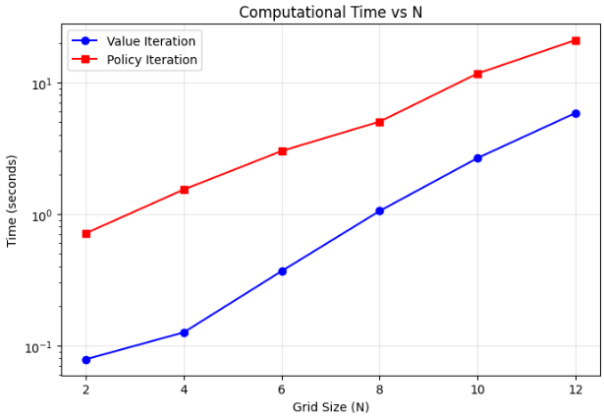


Figure 2: Comparison of execution times.

The L_1-difference plot shows that both methods converge to the same global optimum. The runtime graph highlights that while the state space grows exponentially, our optimizations keep computation manageable for $N = 12$.